



CS 2568

รายงานทางวิชาการในหัวข้อ

การวิเคราะห์ข้อมูลระดับความรุนแรงของอุบัติเหตุประเทศสหรัฐอเมริกา

Analysis of Traffic Accident Severity in the United State

เสนอ

อ. ดร.พบพร ด่านวิรุฑัย

โดย

663380208-9 นายณัฐพงศ์ เหลื่อมล้ำ

663380209-7 นางสาวณิกานต์ จูประพัทธศรี

663380214-4 นายนราวิชญ์ ศิริพล

รายงานนี้เป็นส่วนหนึ่งของการศึกษาวิชา CP353112 การวิเคราะห์ข้อมูลขนาดใหญ่

ภาคเรียนที่ 2 ปีการศึกษา 2569

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น

บทที่ 1 บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

อุบัติเหตุทางถนนเป็นหนึ่งในอุบัติเหตุร้ายแรงที่ส่งผลกระทบต่อชีวิตและทรัพย์สินของประชากรโลกอย่างมหาศาล โดยเฉพาะในสหรัฐอเมริกาที่มีโครงข่ายการคมนาคมที่สลับซับซ้อนและมีความหลากหลายทางภูมิศาสตร์สูง การทำความเข้าใจถึงปัจจัยที่มีอิทธิพลต่อระดับความรุนแรงของอุบัติเหตุ (Accident Severity) จึงมีความสำคัญอย่างยิ่งต่อการวางแผนเชิงยุทธศาสตร์เพื่อลดความสูญเสีย

โครงการนี้จึงมุ่งเน้นการวิเคราะห์ข้อมูลอุบัติเหตุจราจรในสหรัฐอเมริกา (US Traffic Accidents) เพื่อค้นหารูปแบบและความสัมพันธ์ระหว่างตัวแปรเชิงบริบทต่าง ๆ ไม่ว่าจะเป็นสภาพอากาศ ช่วงเวลา สถานที่ หรือลักษณะทางกายภาพของถนน

ข้อมูลที่ได้รับจากการวิเคราะห์เชิงลึกและการสร้างแบบจำลองการพยากรณ์ด้วยการเรียนรู้ของเครื่อง (Machine Learning) จะช่วยให้หน่วยงานที่เกี่ยวข้องสามารถระบุพื้นที่และเงื่อนไขที่มีความเสี่ยงสูงได้อย่างแม่นยำ

นำไปสู่การตัดสินใจเชิงนโยบายและการจัดสรรทรัพยากรในการเฝ้าระวังอุบัติเหตุได้อย่างมีประสิทธิภาพสูงสุด

ข้อมูลชุดนี้ถือเป็นข้อมูลที่มีความสำคัญในระดับสูงเนื่องจากการรวบรวมเหตุการณ์อุบัติเหตุครอบคลุมทั่วประเทศในระยะเวลายาวนาน ซึ่งประกอบไปด้วยตัวแปรที่หลากหลายเกินกว่าที่สถิติทั่วไปจะครอบคลุมได้

ข้อมูลดังกล่าวแสดงให้เห็นถึงปัจจัยที่ควบคุมไม่ได้แต่ส่งผลกระทบอย่างมีนัยสำคัญ เช่น สภาพอากาศที่แปรปรวน ความชื้น และทัศนวิสัย รวมถึงมิติด้านเวลาที่สัมพันธ์กับพฤติกรรมการเดินทางของผู้คนในแต่ละภูมิภาค

การวิเคราะห์ข้อมูลเหล่านี้จึงช่วยให้เราเห็นภาพรวมของความเสี่ยงในเชิงโครงสร้าง และสามารถคาดการณ์ปัจจัยสำคัญที่เป็นตัวแปรต้นเหตุของความรุนแรงในระดับต่าง ๆ ได้อย่างเป็นระบบ

ซึ่งเป็นประโยชน์อย่างมากต่อการกำหนดมาตรการด้านความปลอดภัยสาธารณะและการจัดการระบบจราจรในอนาคต

ความจำเป็นในการนำเทคนิคและเครื่องมือทางด้าน Big Data มาประยุกต์ใช้ในโครงการนี้

เกิดจากลักษณะเฉพาะของข้อมูลที่มีขนาดใหญ่เกินขีดจำกัดของซอฟต์แวร์จัดการเอกสารทั่วไปอย่าง Microsoft Excel โดยมีจำนวนข้อมูลมหาศาลมากกว่า 7.7 ล้านแถว และมีตัวแปรประกอบถึง 46 คอลัมน์ ซึ่งต้องอาศัยการจัดเก็บในรูปแบบไฟล์ Parquet และการประมวลผลผ่าน Library เฉพาะทางอย่าง Pandas และ PyArrow เพื่อให้สามารถจัดการข้อมูลได้อย่างรวดเร็ว

นอกจากนี้ ข้อมูลยังมีความหลากหลายสูง

ทั้งข้อมูลรูปแบบโครงสร้างที่เป็นตัวเลขและข้อมูลข้อความอธิบายเหตุการณ์ที่ต้องใช้กระบวนการ Feature Engineering ที่ซับซ้อนเพื่อสกัดคุณลักษณะสำคัญออกมา การประมวลผลข้อมูลที่มีความซับซ้อนและต้องการความแม่นยำสูงเช่นนี้

จำเป็นต้องอาศัยเทคโนโลยี Big Data และการประมวลผลเชิงคำนวณขั้นสูง เพื่อให้การสร้างแบบจำลอง Machine Learning สามารถดำเนินไปได้อย่างมีประสิทธิภาพและสามารถนำผลลัพธ์ไปใช้งานจริงได้ในสเกลระดับประเทศ

1.2 วัตถุประสงค์ของการศึกษา

- 1 เพื่อวิเคราะห์และระบุปัจจัยเชิงบริบทที่มีอิทธิพลต่อระดับความรุนแรงของอุบัติเหตุ:
มุ่งเน้นการค้นหาคำตอบว่าตัวแปรด้านสภาพแวดล้อม (เช่น อุณหภูมิ, ปริมาณน้ำฝน, ทิศนวิสัย) และลักษณะทางกายภาพของพื้นที่ (Area Density) มีความสัมพันธ์อย่างไรกับระดับความรุนแรงของอุบัติเหตุในแต่ละพื้นที่
- 2 เพื่อศึกษาความสัมพันธ์ระหว่างช่วงเวลากับการเกิดอุบัติเหตุรุนแรง:
เพื่อตอบคำถามว่ารูปแบบการเกิดอุบัติเหตุมีความแตกต่างกันอย่างไรในมิติด้านเวลา เช่น ความแตกต่างระหว่างช่วงกลางวันและกลางคืน โดยใช้เทคนิค Cyclic Transformation ในการประมวลผลข้อมูลเวลา
- 3 เพื่อพัฒนาแบบจำลองการเรียนรู้ของเครื่อง (Machine Learning) ในการพยากรณ์ความรุนแรงของอุบัติเหตุ:
โดยใช้เฉพาะข้อมูลที่ทราบล่วงหน้าก่อนเกิดเหตุ (Pre-incident Features) เพื่อประเมินความแม่นยำและความเป็นไปได้ในการนำโมเดลไปใช้เป็นระบบเตือนภัยหรือสนับสนุนการตัดสินใจของเจ้าหน้าที่
- 4 เพื่อจัดลำดับความสำคัญของตัวแปร (Feature Importance) ที่ส่งผลต่ออุบัติเหตุ: เพื่อระบุว่าตัวแปรใด (เช่น Is_Critical_Description หรือลักษณะถนน) เป็นปัจจัยชี้วัดหลัก (Key Predictors) ที่ส่งผลต่อความรุนแรงระดับสูงสุด เพื่อให้หน่วยงานที่เกี่ยวข้องสามารถวางมาตรการป้องกันได้อย่างตรงจุด
- 5 เพื่อสกัดองค์ความรู้เชิงลึก (Actionable Insights) จากข้อมูลขนาดใหญ่: เพื่อนำผลลัพธ์ที่ได้จากการทำ Focused EDA มาสรุปเป็นแนวทางในการกำหนดนโยบายความปลอดภัยทางถนน และการบริหารจัดการทรัพยากรด้านการจราจรให้สอดคล้องกับพฤติกรรมที่เกิดอุบัติเหตุจริง

1.3 ขอบเขตของโครงการ

1. แหล่งที่มาของข้อมูล (Data Source):
 - ใช้ชุดข้อมูลอุบัติเหตุจราจรในสหรัฐอเมริกา (US Traffic Accidents) จากแพลตฟอร์ม Kaggle ซึ่งเป็นข้อมูลสาธารณะที่รวบรวมข้อมูลอุบัติเหตุแบบ Real-time โดยครอบคลุมพื้นที่ 49 รัฐในสหรัฐอเมริกา

- ช่วงเวลาของข้อมูล (Data Timeline):
 - ข้อมูลที่นำมาศึกษาครอบคลุมเหตุการณ์อุบัติเหตุตั้งแต่ปี 2016 ถึงปี 2023 เพื่อให้เห็นแนวโน้มและการเปลี่ยนแปลงของพฤติกรรมอุบัติเหตุในระยะยาวและครอบคลุมทุกสภาพฤดูกาล
2. ปริมาณข้อมูล (Data Volume):
 - ข้อมูลขนาดใหญ่จำนวนกว่า 7.7 ล้านแถว (7,728,394 rows) และประกอบด้วยตัวแปรตั้งต้นจำนวน 46 คอลัมน์
 3. เทคโนโลยีและเครื่องมือที่ใช้ (Technologies & Tools):
 - ภาษาที่ใช้: Python (เป็นภาษาหลักในการประมวลผลข้อมูลและสร้างโมเดล)
 - การจัดการข้อมูลขนาดใหญ่: ใช้เทคนิคการจัดเก็บข้อมูลในรูปแบบไฟล์ Parquet และประมวลผลผ่าน Library PyArrow ร่วมกับ FastParquet เพื่อประสิทธิภาพในการอ่าน-เขียนข้อมูลที่เร็วกว่าไฟล์ CSV ทั่วไป
 4. การประมวลผลและการทำความสะอาดข้อมูล:
 - ใช้ Pandas และ NumPy สำหรับกระบวนการ ETL (Extract, Transform, Load) และการทำ Feature Engineering
 5. การสร้างแบบจำลองการเรียนรู้ของเครื่อง (Machine Learning):
 - ใช้ Library Scikit-Learn และโมเดลกลุ่ม Tree-based เช่น CatBoost หรือ LightGBM (ตามที่ระบุในไฟล์ 05_modeling.ipynb) ซึ่งเหมาะสำหรับการประมวลผลข้อมูลที่มีความซับซ้อนและมีขนาดใหญ่
 6. การจัดทำรายงานและการแสดงผล (Visualization):
 - ใช้ Matplotlib และ Seaborn ในการวิเคราะห์ข้อมูลเชิงสำรวจ (EDA) เพื่อนำเสนอ Insights ในรูปแบบกราฟและแผนภาพ
 7. ขอบเขตด้านเนื้อหา (Content Scope):
 - ศึกษาเฉพาะปัจจัยที่ทราบก่อนเกิดเหตุ (Pre-incident factors) เช่น สภาพอากาศ, เวลา, และลักษณะพื้นที่
 - จำแนกระดับความรุนแรงของอุบัติเหตุ (Severity) ออกเป็น 4 ระดับ (Multiclass Classification)
 - วิเคราะห์ความสัมพันธ์เชิงพื้นที่และเวลา (Spatial and Temporal Analysis) โดยเน้นไปที่การทำ Cyclic Transformation เพื่อให้โมเดลเข้าใจวัฏจักรของเวลาได้อย่างถูกต้อง

1.4 ประโยชน์ที่คาดว่าจะได้รับ ใครจะได้ประโยชน์จากการวิเคราะห์ครั้งนี้

1.4.1 หน่วยงานภาครัฐและผู้วางนโยบายด้านความปลอดภัย (Government & Policy Makers)

- การวางแผนเชิงป้องกัน: หน่วยงานอย่างกรมการขนส่งหรือกระทรวงคมนาคม สามารถใช้ข้อมูล "ปัจจัยชี้วัดความรุนแรง" ไปกำหนดนโยบายลดอุบัติเหตุให้ตรงจุด เช่น การปรับปรุงมาตรฐานถนนในจุดที่โมเดลระบุว่าพื้นที่ความหนาแน่นสูง (High Density) และมีสถิติความรุนแรงระดับ 4 บ่อยครั้ง
- การจัดสรรทรัพยากร:
ช่วยให้หน่วยงานกู้ภัยและตำรวจจราจรสามารถวางกำลังเจ้าหน้าที่หรือเตรียมพร้อมอุปกรณ์ในจุดเสี่ยง ตามช่วงเวลา (Time) และสภาพอากาศ (Weather) ที่โมเดลพยากรณ์ว่าจะเกิดอุบัติเหตุรุนแรง

1.4.2 หน่วยงานด้านการจัดการจราจรและผังเมือง (Traffic Management & Urban Planning)

- การปรับปรุงโครงสร้างพื้นฐาน: ข้อมูลจาก Feature Importance จะช่วยให้วิศวกรจราจรเห็นความสำคัญของลักษณะทางกายภาพถนน (เช่น จุดทางแยก หรือจุดที่มีป้ายหยุด) ว่าส่งผลต่อความรุนแรงเพียงใด เพื่อนำไปสู่การออกแบบผังเมืองและระบบป้ายสัญญาณที่มีประสิทธิภาพมากขึ้น
- ระบบเตือนภัยอัจฉริยะ (Smart Warning Systems): สามารถนำโมเดลการพยากรณ์ไปรวมเข้ากับระบบนำทาง เพื่อแจ้งเตือนผู้ขับขี่ล่วงหน้าเมื่อขับเข้าสู่สถานะที่มีความเสี่ยงสูง (เช่น ช่วงเวลาที่มีทัศนวิสัยต่ำร่วมกับสภาพถนนที่เปียกชื้น)

1.4.3 ผู้ใช้รถใช้ถนนและภาคประชาชน (Public & Drivers)

- การตระหนักรู้ด้านความเสี่ยง: ผลการวิเคราะห์ (Insights) จะช่วยให้ประชาชนเข้าใจถึงความเสี่ยงของตนเองในบริบทต่าง ๆ เช่น การขับขี่ในสภาพอากาศที่เฉพาะเจาะจง ซึ่งจะช่วยกระตุ้นให้เกิดพฤติกรรมการขับขี่ที่ระมัดระวังมากขึ้น
- ความปลอดภัยที่เพิ่มขึ้น: ในระยะยาว ประชาชนจะได้รับประโยชน์จากการที่อัตราการเกิดอุบัติเหตุรุนแรงลดลง เนื่องจากการปรับปรุงแก้ไขปัญหาเชิงรุกของหน่วยงานที่เกี่ยวข้อง

1.4.4 วงการวิชาการและนักวิเคราะห์ข้อมูล (Academic & Data Science Community)

- ต้นแบบการวิเคราะห์ Big Data: โครงการนี้เป็นกรณีศึกษา (Use Case) ในการประมวลผลข้อมูลขนาดหลักล้านชุดด้วยเทคนิคสมัยใหม่ เช่น การใช้ไฟล์ Parquet, การทำ Cyclic Transformation สำหรับข้อมูลเวลา และการจัดการกับข้อมูลที่ไม่สมดุล (Imbalanced Data) ซึ่งสามารถนำไปต่อยอดในการศึกษาด้านความปลอดภัยอื่น ๆ ได้

บทที่ 2 เอกสารและงานวิจัยที่เกี่ยวข้อง

2.1 แนวคิดเกี่ยวกับ Big Data และหลักการ 5Vs

ในการวิเคราะห์ข้อมูลอุบัติเหตุที่มีจำนวนมากกว่า 7.7 ล้านแถว จำเป็นต้องเข้าใจคุณลักษณะของ Big Data ผ่านหลักการ 5Vs ซึ่งสอดคล้องกับชุดข้อมูลที่ใช้ ดังนี้

- **Volume (ขนาด):** ข้อมูลมีปริมาณมหาศาลเกินกว่าที่หน่วยความจำของโปรแกรมทั่วไปจะรับไหว
- **Velocity (ความเร็ว):** ข้อมูลอุบัติเหตุมีการอัปเดตและต้องประมวลผลแข่งกับเวลาเพื่อการแจ้งเตือน
- **Variety (ความหลากหลาย):** มีทั้งข้อมูลตัวเลข (สภาพอากาศ), ข้อมูลเชิงหมวดหมู่ (รัฐ/เมือง), และข้อมูลข้อความ (Description)
- **Veracity (ความถูกต้อง):** กระบวนการ Data Cleaning ในไฟล์ที่ 2 มุ่งเน้นการจัดการ Missing Values และ Outliers เพื่อคงความน่าเชื่อถือของข้อมูล
- **Value (คุณค่า):** การสกัด Insight เพื่อนำไปสู่การลดอัตราการเสียชีวิตบนท้องถนน

2.2 เครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนา

โครงงานนี้เลือกใช้ Ecosystem ของ Python ในการจัดการ Big Data แทนเครื่องมือดั้งเดิม:

- **Apache Arrow & Parquet:** เทคโนโลยีการจัดเก็บข้อมูลแบบ Columnar Storage ที่ช่วยให้การอ่านข้อมูลขนาด 7 ล้านแถวทำได้รวดเร็วและใช้พื้นที่น้อยกว่า CSV หลายเท่า
- **Pandas & NumPy:** เครื่องมือหลักในการทำ Data Manipulation และกระบวนการ ETL (Extract, Transform, Load) เพื่อเตรียมข้อมูลให้พร้อมสำหรับการวิเคราะห์
- **Scikit-Learn & Tree-based Models:** การเลือกใช้ไลบรารีสำหรับการทำ Machine Learning โดยเฉพาะโมเดลกลุ่ม Decision Tree (เช่น Random Forest หรือ Gradient Boosting) ที่มีประสิทธิภาพสูงในการจัดการกับข้อมูลที่มีความสัมพันธ์ซับซ้อน

2.3 ทฤษฎีด้าน Data Analytics และ Machine Learning

- **Exploratory Data Analysis (EDA):** แนวคิดการสำรวจข้อมูลเพื่อค้นหารูปแบบ (Pattern) และความผิดปกติ โดยในโครงงานนี้มีการใช้ Focused EDA เพื่อเจาะจงปัจจัยที่ส่งผลต่อความรุนแรง (Severity) โดยเฉพาะ

- **Feature Engineering:** ทฤษฎีการสร้างตัวแปรใหม่เพื่อให้โมเดลเรียนรู้ได้ดีขึ้น เช่น Cyclic Transformation (แปลงเวลาเป็นค่า Sin/Cos) เพื่อรักษาคุณสมบัติของคาบเวลา และการทำ Area Density Classification
- **Classification Theory:** เนื่องจากการพยากรณ์ความรุนแรงของอุบัติเหตุ (ระดับ 1-4) เป็นปัญหาแบบ Multiclass Classification จึงต้องใช้ทฤษฎีการวัดผลด้วย Confusion Matrix, Precision, Recall และ F1-Score ในการตัดสินประสิทธิภาพของโมเดล

2.4 งานวิจัยและโครงการที่เกี่ยวข้อง

จากการศึกษาแนวทางการวิเคราะห์ชุดข้อมูล US Traffic Accidents ในแพลตฟอร์ม Kaggle และงานวิจัยด้านวิศวกรรมจราจร พบว่า:

- **การจัดการ Data Leakage:** งานวิจัยส่วนใหญ่ระบุว่าต้องตัดตัวแปรที่เกิดขึ้น "หลัง" อุบัติเหตุออก (เช่น ระยะเวลาที่ปิดถนน) เพื่อให้โมเดลสามารถพยากรณ์ล่วงหน้าได้จริง
- **อิทธิพลของสภาพอากาศ:** งานวิจัยในอดีตยืนยันว่า "ทัศนวิสัย" (Visibility) และ "หยาดน้ำฟ้า" (Precipitation) เป็นปัจจัยลำดับต้น ๆ ที่ส่งผลต่ออุบัติเหตุรุนแรง ซึ่งโครงการนี้ได้นำมาทดสอบผ่าน Feature Importance ในไฟล์ที่ 5
- **ความไม่สมดุลของข้อมูล (Imbalanced Data):** เนื่องจากอุบัติเหตุรุนแรงมาก (Severity 4) มักมีจำนวนน้อยกว่าอุบัติเหตุทั่วไป
งานวิจัยที่เกี่ยวข้องจึงแนะนำให้ใช้เทคนิคการสุ่มตัวอย่างหรือโมเดลที่ทนทานต่อข้อมูล Imbalanced เช่น CatBoost

บทที่ 3: วิธีการดำเนินงาน (Research Methodology)

ในการศึกษาครั้งนี้ ผู้จัดทำได้ดำเนินการวิเคราะห์ข้อมูลอุบัติเหตุจราจรในสหรัฐอเมริกา (US Traffic Accidents) โดยแบ่งขั้นตอนการดำเนินงานออกเป็น 5 ขั้นตอนหลัก ตามรายละเอียดดังนี้

3.1 การรวบรวมและจัดเตรียมข้อมูลตั้งต้น (Data Collection & Initial Loading)

ในขั้นตอนเริ่มต้นของการศึกษา โครงการนี้ได้นำชุดข้อมูลอุบัติเหตุทางถนนในสหรัฐอเมริกา (US Traffic Accidents) ซึ่งเป็นข้อมูลขนาดใหญ่ (Big Data) มาดำเนินการจัดเตรียมเพื่อใช้งาน โดยมีรายละเอียดขั้นตอนทางเทคนิคดังนี้

3.1.1 การติดตั้งสภาพแวดล้อมและ Library ที่จำเป็น

เนื่องจากข้อมูลมีขนาดใหญ่กว่า 7.7 ล้านแถว การใช้ Library มาตรฐานเพียงอย่างเดียวอาจไม่เพียงพอ จึงมีการเลือกใช้เทคโนโลยีที่รองรับการประมวลผลประสิทธิภาพสูง

```
%pip install pandas matplotlib seaborn numpy pyarrow fastparquet
```

ภาพที่ 1 : การติดตั้งและนำเข้า Library สำคัญ

อธิบายภาพที่ 1 : ดำเนินการติดตั้งและนำเข้า Library สำคัญ ได้แก่ Pandas สำหรับการจัดการโครงสร้างข้อมูล, PyArrow และ FastParquet ซึ่งเป็น Engine สำคัญในการอ่านไฟล์ข้อมูลรูปแบบ Columnar Storage เพื่อให้การโหลดข้อมูลหลักล้านแถวทำได้รวดเร็ว กว่าการใช้ไฟล์ CSV ทั่วไปหลายเท่า

3.1.2 การโหลดข้อมูลดิบและตรวจสอบโครงสร้างเบื้องต้น

การโหลดข้อมูลที่มีจำนวนคอลัมน์ถึง 46 คอลัมน์ จำเป็นต้องตรวจสอบความถูกต้องของประเภทข้อมูล (Data Types) ตั้งแต่เริ่มต้น

```

import pandas as pd
import os

# -----
# 1) ตั้งค่าให้ Pandas แสดงผลอ่านง่าย
# -----
pd.set_option('display.max_columns', None) # แสดงทุกคอลัมน์
pd.set_option('display.width', 200) # ความกว้างตาราง
pd.set_option('display.max_colwidth', 20) # ความกว้างข้อความต่อคอลัมน์
pd.set_option('display.float_format', '{:.4f}'.format) # ทศนิยมสวย ๆ

# -----
# 2) path ไปยังไฟล์ CSV
# -----
file_path = r"C:\punpun\Big data\US-Traffic-Accident-BigData-Analysis\data\raw\US_Accidents_March23.csv"

# -----
# 3) โหลดข้อมูล
# -----
if os.path.exists(file_path):
    print("พบไฟล์แล้ว กำลังโหลดข้อมูล...\n")

    df = pd.read_csv(file_path)

    print(f"โหลดสำเร็จ! จำนวนข้อมูล: {len(df):,} แถว")
    print(f"จำนวนคอลัมน์: {df.shape[1]}\n")

    # แสดง 5 แถวแรก แบบเห็นครบทุกคอลัมน์
    display(df.head())
else:
    print(f"ไม่พบไฟล์ที่: {os.path.abspath(file_path)}")

```

ภาพที่ 2 : การดึงข้อมูลจากไฟล์ดิบเข้าสู่ตัวแปร

อธิบายภาพที่ 2 : เป็นการดึงข้อมูลจากไฟล์ดิบเข้าสู่ตัวแปร DataFrame โดยระบบจะรายงานจำนวนแถวและคอลัมน์ทั้งหมดทันที เพื่อ ยืนยันความครบถ้วนของข้อมูลก่อนนำไปประมวลผลต่อ โดยพบว่ามีจำนวนข้อมูล 7,728,394 แถว จำนวนคอลัมน์ 46 คอลัมน์

```
df.info()

<class 'pandas.DataFrame'>
RangeIndex: 7728394 entries, 0 to 7728393
Data columns (total 46 columns):
#   Column              Dtype
---  -
0   ID                  str
1   Source              str
2   Severity            int64
3   Start_Time         str
4   End_Time           str
5   Start_Lat          float64
6   Start_Lng          float64
7   End_Lat            float64
8   End_Lng            float64
9   Distance(mi)       float64
10  Description         str
11  Street             str
12  City               str
13  County             str
14  State              str
15  Zipcode            str
16  Country            str
17  Timezone           str
18  Airport_Code       str
19  Weather_Timestamp  str
...
44  Nautical_Twilight  str
45  Astronomical_Twilight str
dtypes: bool(13), float64(12), int64(1), str(20)
memory usage: 3.6 GB
```

ภาพที่ 3 : ตรวจสอบความถูกต้องของประเภทข้อมูล

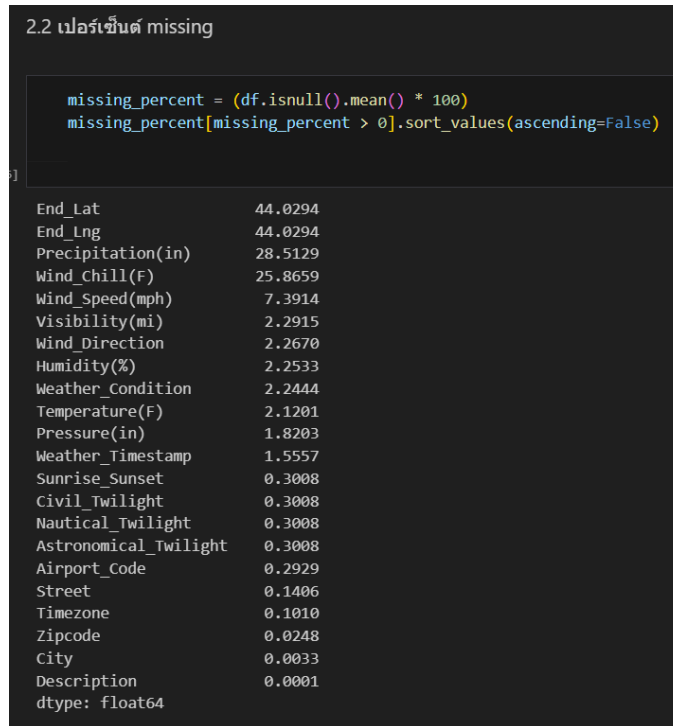
อธิบายภาพที่ 3 : ใช้คำสั่ง df.info

เพื่อตรวจสอบประเภทข้อมูลของแต่ละคอลัมน์เพื่อตรวจสอบว่าสามารถแบ่งกลุ่มตามประเภทได้อย่างไรบ้าง เพื่อเตรียมพร้อมสำหรับการแก้ไขข้อมูลบางคอลัมน์ที่ผิด และการจัดกลุ่ม Column ตามประเภทที่ถูกต้อง

3.1.3 การสำรวจค่าสูญหาย (Missing Value Analysis)

เนื่องจากข้อมูล Big Data มักมาจากหลายแหล่ง (Multiple Sources) ปัญหา Missing Values

จึงเป็นเรื่องปกติที่ต้องระบุจำนวนให้ชัดเจน



ภาพที่ 4 : ตรวจสอบความถูกต้องของประเภทข้อมูล

อธิบายภาพที่ 4 : ดำเนินการนับจำนวนค่าว่างในทุกคอลัมน์และคำนวณเป็นเปอร์เซ็นต์

เพื่อระบุว่าตัวแปรใดที่มีข้อมูลไม่เพียงพอต่อการวิเคราะห์ (เช่น คอลัมน์ End_Lat หรือ Precipitation ในช่วงปีแรกๆ)

ซึ่งข้อมูลส่วนนี้จะถูกส่งต่อไปยังกระบวนการ Cleaning เพื่อตัดสินใจว่าจะ "ตัดออก" หรือ "เติมค่า" (Impute)

3.1.4 การวิเคราะห์สถิติพรรณนาและความผิดปกติ (Descriptive Statistics & Logic Validity)

การตรวจสอบค่าทางสถิติเพื่อดูการกระจายตัวของข้อมูล และหาค่าที่ "เป็นไปได้" ทางตรรกะ เราหาค่า Variance เพื่อบอกเราว่าข้อมูลแต่ละจุดอยู่ห่างจากค่าเฉลี่ยมากน้อยแค่ไหน ถ้าค่าสูง แสดงว่าข้อมูลมีความหลากหลาย (Diverse)

แต่ถ้าค่าต่ำมากหรือเป็น 0 แสดงว่าข้อมูลแทบจะเหมือนกันหมด หากตัวแปรใดมี Variance ต่ำมาก (เกือบเป็น 0)

หมายความว่าตัวแปรนั้นไม่มีพลังในการจำแนกข้อมูล เช่น ถ้าทุกเคสใน Dataset มีค่า Precipitation เป็น 0.01 เท่ากันหมด

ตัวแปรนี้ก็ไม่ช่วยให้โมเดลทำนายได้ว่าอุบัติเหตุจะแรงหรือไม่แรง การหา Outliers เพียงไม่กี่ค่าสามารถดึงค่าเฉลี่ย (Mean)

และค่าความแปรปรวน (Variance) ให้สูงผิดปกติ ทำให้เราเข้าใจภาพรวมของ Dataset ผิดไป

และช่วยให้เห็นว่าข้อมูลที่ผิดปกติเกิดจากความผิดพลาดในการบันทึก (เช่น อุณหภูมิ 500 องศา)

หรือเป็นเหตุการณ์ที่เกิดขึ้นจริงแต่หาได้ยาก

```
pd.set_option('display.max_rows', None)
num_summary = df[num_cols].describe().T
num_summary
```

	count	mean	std	min	25%	50%	75%	max
Severity	7728394.0000	2.2124	0.4875	1.0000	2.0000	2.0000	2.0000	4.0000
Start_Lat	7728394.0000	36.2012	5.0761	24.5548	33.3996	35.8240	40.0850	49.0022
Start_Lng	7728394.0000	-94.7025	17.3918	-124.6238	-117.2194	-87.7666	-80.3537	-67.1132
End_Lat	4325632.0000	36.2618	5.2729	24.5660	33.4621	36.1835	40.1789	49.0750
End_Lng	4325632.0000	-95.7256	18.1079	-124.5457	-117.7543	-88.0279	-80.2471	-67.1092
Distance(mi)	7728394.0000	0.5618	1.7768	0.0000	0.0000	0.0300	0.4640	441.7500
Temperature(F)	7564541.0000	61.6633	19.0137	-89.0000	49.0000	64.0000	76.0000	207.0000
Wind_Chill(F)	5729375.0000	58.2510	22.3898	-89.0000	43.0000	62.0000	75.0000	207.0000
Humidity(%)	7554250.0000	64.8310	22.8210	1.0000	48.0000	67.0000	84.0000	100.0000
Pressure(in)	7587715.0000	29.5390	1.0062	0.0000	29.3700	29.8600	30.0300	58.6300
Visibility(mi)	7551296.0000	9.0904	2.6883	0.0000	10.0000	10.0000	10.0000	140.0000
Wind_Speed(mph)	7157161.0000	7.6855	5.4250	0.0000	4.6000	7.0000	10.4000	1087.0000
Precipitation(in)	5524808.0000	0.0084	0.1102	0.0000	0.0000	0.0000	0.0000	36.4700

ภาพที่ 5 : ตรวจสอบค่าทางสถิติเพื่อดูการกระจายตัวของข้อมูล

อธิบายภาพที่ 5 : ใช้คำสั่งสถิติพรรณนาเพื่อดูค่า Mean, Std และ Percentiles ของตัวแปรเชิงปริมาณ

นอกจากนี้จากผลลัพธ์เราพบข้อมูลหลายอย่างที่น่าสนใจ เช่น พบประเด็นสำคัญในคอลัมน์ Distance(mi) ที่มีค่าสูงสุดถึง 441 ไมล์ ซึ่งขัดต่อหลักตรรกะของจุดเกิดอุบัติเหตุ จึงต้องดูค่า Percentile ที่ 99.9 เพื่อพิจารณาว่าควรจะทำการ Clip ข้อมูลที่จุดใด เพื่อไม่ให้ค่าที่ผิดปกติเหล่านี้ไปดึงค่ากลางของโมเดลให้คลาดเคลื่อน

```
variance = df[num_cols].var().sort_values(ascending=False)
variance
```

Humidity(%)	520.7966
Wind_Chill(F)	501.3046
Temperature(F)	361.5190
End_Lng	327.8970
Start_Lng	302.4732
Wind_Speed(mph)	29.4304
End_Lat	27.8035
Start_Lat	25.7666
Visibility(mi)	7.2270
Distance(mi)	3.1571
Pressure(in)	1.0124
Severity	0.2377
Precipitation(in)	0.0121

dtype: float64

ภาพที่ 6 : ตรวจสอบความแปรปรวนของข้อมูล

อธิบายภาพที่ 6 : ข้อมูลมีความหลากหลายมากในเรื่อง สภาพอากาศ (Humidity, Wind Chill, Temp) แต่มีความสม่ำเสมอสูงมากในเรื่อง ปริมาณฝน (Precipitation) และ ระดับความรุนแรง (Severity) โดย Variance ที่ต่ำขนาดนี้อาจหมายความว่าเราแทบจะใช้ข้อมูลฝนมาแยกแยะอะไรไม่ได้เลย

```
def iqr_outlier(df, col):
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    outlier_count = ((df[col] < lower) | (df[col] > upper)).sum()

    return pd.Series({
        'Q1': Q1,
        'Q3': Q3,
        'IQR': IQR,
        'Lower': lower,
        'Upper': upper,
        'Outlier_Count': outlier_count,
        'Outlier_%': outlier_count / len(df) * 100
    })

outlier_df = df[important_num_cols].apply(lambda x: iqr_outlier(df, x.name)).T
outlier_df
```

	Q1	Q3	IQR	Lower	Upper	Outlier_Count	Outlier_%
Severity	2.0000	2.0000	0.0000	2.0000	2.0000	1571413.0000	20.3330
Distance(mi)	0.0000	0.4640	0.4640	-0.6960	1.1600	963606.0000	12.4684
Temperature(F)	49.0000	76.0000	27.0000	8.5000	116.5000	50515.0000	0.6536
Humidity(%)	48.0000	84.0000	36.0000	-6.0000	138.0000	0.0000	0.0000
Pressure(in)	29.3700	30.0300	0.6600	28.3800	31.0200	443383.0000	5.7371
Visibility(mi)	10.0000	10.0000	0.0000	10.0000	10.0000	1481065.0000	19.1639
Wind_Speed(mph)	4.6000	10.4000	5.8000	-4.1000	19.1000	210603.0000	2.7251
Precipitation(in)	0.0000	0.0000	0.0000	0.0000	0.0000	533090.0000	6.8978

ภาพที่ 7 : การหาค่า Outliers โดยใช้ระเบียบวิธี Interquartile Range

อธิบายภาพที่ 7 : จากการวิเคราะห์ค่าความผิดปกติ (Outliers) ด้วยวิธี IQR พบว่าตัวแปรประเภทกลุ่มหรือค่าไม่ต่อเนื่อง เช่น **Severity** และ **Visibility** มีสัดส่วน Outliers สูงผิดปกติ (ประมาณร้อยละ 19-20)

เนื่องจากข้อมูลส่วนใหญ่กระจุกตัวอยู่ที่ค่าเดียว ทำให้ค่าอื่นถูกมองเป็นค่าผิดปกติทั้งหมดตามสูตรคำนวณ

ในขณะที่ตัวแปรด้านสภาพอากาศอย่าง **Temperature** และ **Wind Speed** มีการกระจายตัวที่เหมาะสมและมีค่าผิดปกติน้อย (ต่ำกว่าร้อยละ 3) สะท้อนถึงคุณภาพข้อมูลที่ค่อนข้างสะอาด อย่างไรก็ตาม พบปัญหา Zero Variance ในตัวแปร **Precipitation** ที่ข้อมูลกว่าร้อยละ 75 เป็นศูนย์ ส่งผลให้การมีฝนตกเพียงเล็กน้อยถูกนับเป็น Outlier ทันที ดังนั้น ทีม Data Analyst

จึงไม่ควรตัดข้อมูลทิ้งเพียงเพราะเกณฑ์ทางสถิติ $1.5 \times IQR$ แต่ควรพิจารณาใช้แผนภาพกล่อง (Box Plot)

ร่วมกับดุลยพินิจในเชิงบริบทของอุบัติเหตุ เพื่อรักษาข้อมูลสำคัญที่มีผลต่อการสร้างแบบจำลองในบทที่ 4

ให้มีความแม่นยำและน่าเชื่อถือสูงสุด

3.1.5 การตรวจสอบความสัมพันธ์ระหว่างตัวแปร (Correlation Analysis)

การวิเคราะห์ความสัมพันธ์ของการหายไปของข้อมูล (Missingness Correlation) การตรวจสอบว่าข้อมูลที่หายไปนั้นมีรูปแบบ (Pattern) หรือมีสาเหตุมาจากตัวแปรอื่นหรือไม่ ซึ่งเป็นขั้นตอนที่สำคัญมากเพื่อใช้ในการทำ Data Cleansing และ Modeling

```
# 1. รันฟังก์ชันที่ปรับปรุงใหม่
missing_corr = compute_missing_correlations_fast(df, missing_cols, numeric_for_analysis, datetime_num_cols)

# 2. แสดงผล Top 5 ความสัมพันธ์ที่อาจทำให้เกิดการหายไปของข้อมูล
for col, correlations in missing_corr.items():
    print(f"\n--- Top correlations for missingness in: {col} ---")
    # กรองเฉพาะที่มีนัยสำคัญ (เช่น > 0.05 หรือ < -0.05)
    top_corr = correlations[correlations.abs() > 0.05].head(5)
    if not top_corr.empty:
        print(top_corr)
    else:
        print("Likely MCAR (Missing Completely At Random) - No strong correlations found.")
```

```
--- Top correlations for missingness in: End_Lng ---
Distance(mi)          -0.7104
Accident_Duration_Min -0.5115
Pressure(in)           0.1704
End_Hour              -0.1077
End_Weekday           -0.1074
dtype: float64

--- Top correlations for missingness in: End_Lat ---
Distance(mi)          -0.7104
Accident_Duration_Min -0.5115
Pressure(in)           0.1704
End_Hour              -0.1077
End_Weekday           -0.1074
dtype: float64

--- Top correlations for missingness in: Precipitation(in) ---
Pressure(in)           0.4389
Accident_Duration_Min -0.2529
Wind_Chill(F)          -0.2473
Distance(mi)           -0.2387
Wind_Speed(mph)        0.0888
dtype: float64
...
Likely MCAR (Missing Completely At Random) - No strong correlations found.

--- Top correlations for missingness in: Description ---
Likely MCAR (Missing Completely At Random) - No strong correlations found.
```

ภาพที่ 7 : การวิเคราะห์ความสัมพันธ์ของการสูญหายของข้อมูล

อธิบายภาพที่ 7 : เราได้ทำการเลือกมาบางคอลัมน์ที่สำคัญ จากการวิเคราะห์ความสัมพันธ์ของการสูญหายของข้อมูล (Missingness Correlation) พบว่าข้อมูลสามารถจำแนกตามลักษณะการหายไปเป็นสองกลุ่มหลัก คือ กลุ่มที่มีการสูญหายแบบไม่สุ่ม (Missing at Random - MAR) ได้แก่ ตัวแปรพิกัดปลายทาง (End_Lat, End_Lng) ซึ่งมีความสัมพันธ์เชิงลบอย่างมีนัยสำคัญกับระยะทาง (Distance) และระยะเวลาการเกิดอุบัติเหตุ (Accident_Duration_Min) บ่งชี้ว่าอุบัติเหตุระยะสั้นมักขาดการบันทึกพิกัดจุดสิ้นสุด รวมถึงกลุ่มข้อมูลสภาพอากาศ (Precipitation, Wind_Chill) ที่สัมพันธ์กับความกดอากาศ (Pressure) ซึ่งอาจเกิดจากข้อจำกัดของอุปกรณ์วัดในสภาวะเฉพาะ และกลุ่มที่มีการสูญหายแบบสุ่มโดยสมบูรณ์ (Missing Completely at Random - MCAR) เช่น Visibility, Humidity

และข้อมูลที่อยู่ ซึ่งไม่ปรากฏรูปแบบความสัมพันธ์กับตัวแปรอื่น

ทำให้สามารถบริหารจัดการข้อมูลด้วยการลบหรือแทนที่ด้วยค่ากลางได้โดยไม่ส่งผลกระทบต่อความเที่ยงตรงของสถิติในภาพรวม ดังนั้น ทีมผู้วิจัยจึงควรระมัดระวังในการกำจัดข้อมูลกลุ่ม MAR เพื่อป้องกันการเกิดอคติจากการคัดเลือกกลุ่มตัวอย่าง (Selection Bias) ที่อาจส่งผลต่อความแม่นยำของแบบจำลองในขั้นตอนการวิเคราะห์เชิงทำนายต่อไป

3.2 การทำความสะอาดข้อมูล (Data Cleaning)

ในขั้นตอนนี้เป็นการเตรียมความพร้อมของข้อมูลดิบที่มีจำนวนกว่า 7.7 ล้านแถว ให้มีความสมบูรณ์ แม่นยำ และพร้อมสำหรับการนำไปวิเคราะห์ในขั้นสูง การทำความสะอาดข้อมูลเราไม่ได้เติมค่าให้ครบเพียงอย่างเดียว แต่เติมค่าอย่างมีตรรกะ (Logical Imputation) โดยมีรายละเอียดการดำเนินการในแต่ละส่วนดังนี้

3.2.1 การโหลดข้อมูลดิบ (Load Raw Data)

```
import pandas as pd
import os

# -----
# 1) ตั้งค่าให้ Pandas แสดงผลอ่านง่าย
# -----
pd.set_option('display.max_columns', None) # แสดงทุกคอลัมน์
pd.set_option('display.width', 200) # ความกว้างตาราง
pd.set_option('display.max_colwidth', 20) # ความกว้างข้อความต่อคอลัมน์
pd.set_option('display.float_format', '{:.4f}'.format) # ทศนิยมสวย ๆ

# -----
# 2) path ไปยังไฟล์ csv
# -----
file_path = r"C:\punpun\Big_data\US-Traffic-Accident-BigData-Analysis\data\raw\US_Accidents_March23.csv"

# -----
# 3) โหลดข้อมูล
# -----
if os.path.exists(file_path):
    print("พบไฟล์แล้ว กำลังโหลดข้อมูล...\n")

    df = pd.read_csv(file_path)

    print(f"โหลดสำเร็จ! จำนวนข้อมูล: {len(df):,} แถว")
    print(f"จำนวนคอลัมน์: {df.shape[1]}\n")

    # แสดง 5 แถวแรก แบบเห็นครบทุกคอลัมน์
    display(df.head())
else:
    print(f"ไม่พบไฟล์ที่: {os.path.abspath(file_path)}")
```

ภาพที่ 8 : การโหลดข้อมูลดิบ

อธิบายภาพที่ 8 : ทำการโหลดไฟล์ US_Accidents_March23.csv เข้าสู่ระบบด้วย Pandas โดยมีการตั้งค่าการแสดงผลตาราง (Display Option) ให้เหมาะสม เช่น กำหนดจำนวนทศนิยมและขอบเขตการแสดงคอลัมน์เพื่อให้ง่ายต่อการตรวจสอบเบื้องต้น

3.2.2 การจัดรูปแบบวันที่ (Parse Datetime)

เราไม่ได้ทำการลบข้อมูลที่ระบุเวลา (Start_Time) ที่ทับกันที่ เพื่อความแม่นยำในการวิเคราะห์ช่วงเวลา นอกจากนี้การเติมค่าสูญหาย (Missing Values) ของสภาพอากาศ (Temperature, Precipitation) เราได้ใช้ค่ามัธยฐาน (Median) ที่แยกตาม "รัฐ (State)" และ "เดือน (Month)" เพื่อสะท้อนสภาพภูมิอากาศจริงในแต่ละพื้นที่และฤดูกาล เพื่อแก้ไขความขัดแย้งของข้อมูล เช่น กรณีที่สภาพอากาศระบุว่า "ฝนตก" แต่ปริมาณน้ำฝนเป็น Null ให้เติมด้วยค่าเฉลี่ย แทนที่จะเป็น 0

```
datetime_cols = ['Start_Time', 'End_Time', 'Weather_Timestamp']

for col in datetime_cols:
    df[col] = pd.to_datetime(df[col], errors='coerce')

df[datetime_cols].dtypes
```

Start_Time	datetime64[us]
End_Time	datetime64[us]
Weather_Timestamp	datetime64[us]
dtype:	object

ภาพที่ 9 : การจัดรูปแบบวันที่

อธิบายภาพที่ 9 : แปลงคอลัมน์ Start_Time, End_Time, และ Weather_Timestamp จากรูปแบบข้อความ (String) ให้เป็นรูปแบบวันที่และเวลา (Datetime object) เพื่อให้คอมพิวเตอร์สามารถคำนวณทางคณิตศาสตร์เกี่ยวกับเวลาได้ โดยใช้คำสั่ง errors='coerce' เพื่อจัดการกับข้อมูลวันที่ที่รูปแบบผิดพลาดให้กลายเป็นค่าว่าง (NaT) โดยอัตโนมัติเพื่อป้องกัน Error ในภายหลัง

3.2.3 การสร้างคุณลักษณะใหม่จากเวลา (Recreate Time Features)

```
df['Start_Hour'] = df['Start_Time'].dt.hour
df['Start_Weekday'] = df['Start_Time'].dt.weekday
df['Start_Month'] = df['Start_Time'].dt.month

df['End_Hour'] = df['End_Time'].dt.hour
df['End_Weekday'] = df['End_Time'].dt.weekday

df['Is_Weekend'] = df['Start_Weekday'].isin([5, 6]).astype(int)

df['Accident_Duration_Min'] = (
    df['End_Time'] - df['Start_Time']
).dt.total_seconds() / 60
```

ภาพที่ 10 : การสร้างคุณลักษณะใหม่จากเวลา

อธิบายภาพที่ 10 : สกัดข้อมูลย่อยจากคอลัมน์ Start_Time ออกมาเป็นคุณลักษณะใหม่ (Feature Engineering)

ได้แก่:

- Hour, Weekday, Month: เพื่อดูช่วงเวลา ชั่วโมง วันในสัปดาห์ และเดือนที่เกิดอุบัติเหตุ
- Is_Weekend: สร้างตัวแปรบ่งชี้ว่าเป็นวันเสาร์-อาทิตย์หรือไม่ (0 หรือ 1)
- Accident_Duration_Min: คำนวณระยะเวลาการเกิดอุบัติเหตุ (End_Time ลบ Start_Time) ในหน่วยนาที

3.2.4 การจัดการค่าว่างและข้อมูลที่เสียหาย (Handling Missing Values)

```
Precipitation

df['Has_Precipitation'] = df['Precipitation(in)'].notna().astype(int)
df['Precipitation(in)'] = df['Precipitation(in)'].fillna(0)

Wind Chill

df['Wind_Speed(mph)'] = (
    df.groupby('State')['Wind_Speed(mph)']
    .transform(lambda x: x.fillna(x.median()))
)

3.4 Categorical Missing

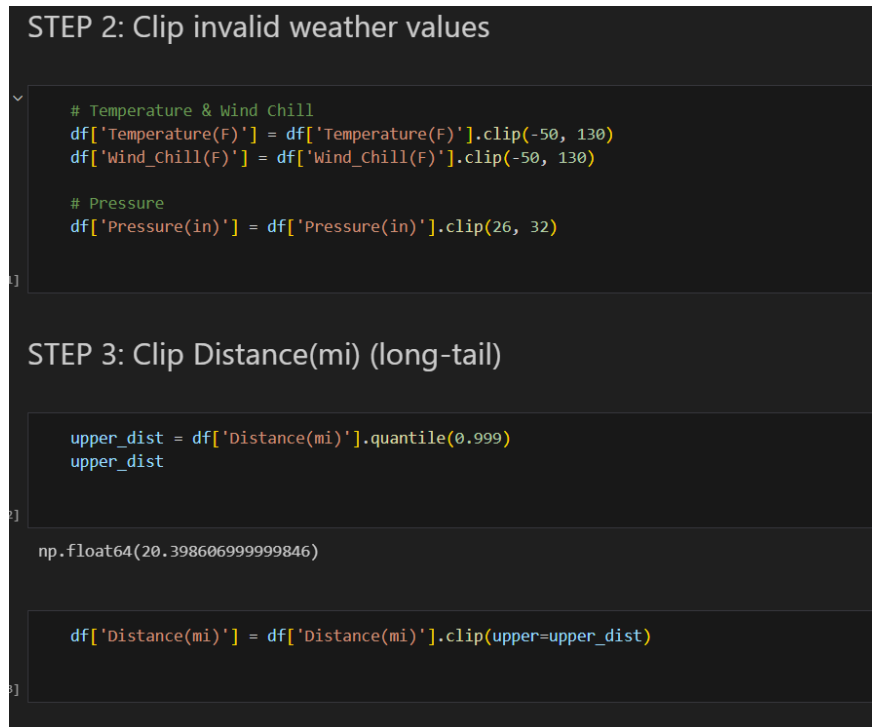
df['Weather_Condition'] = df['Weather_Condition'].fillna('Unknown')
df['Sunrise_Sunset'] = df['Sunrise_Sunset'].fillna('Unknown')
```

ภาพที่ 11 : การจัดการค่าว่างและข้อมูลที่เสียหาย

อธิบายภาพที่ 11 : เป็นการกำจัดช่องว่างในข้อมูล (NaN) ด้วยเทคนิคที่คำนึงถึงความเป็นจริง:

- Start_Time: ลบแถวที่ไม่มีข้อมูลเวลาเริ่มต้นทั้ง (เหลือข้อมูลประมาณ 6.9 ล้านแถว)
- Weather Numeric: เติมค่าว่างในกลุ่มสภาพอากาศ (อุณหภูมิ, ความชื้น, ฯลฯ) โดยใช้ "ค่ามัธยฐาน (Median) ของรัฐนั้นๆ ในเดือนนั้นๆ" เพื่อให้ข้อมูลมีความสมเหตุสมผลตามสภาพภูมิศาสตร์และฤดูกาล
- Precipitation: เติมค่า 0 ในกรณีที่สภาพอากาศระบุว่าฟ้าใสแต่ไม่มีตัวเลขปริมาณน้ำฝน
- Categorical: เติมค่า 'Unknown' ในกลุ่มข้อมูลประเภทตัวอักษรที่ขาดหายไป

3.2.5 การจัดการค่าผิดปกติ (Outliers Clipping)

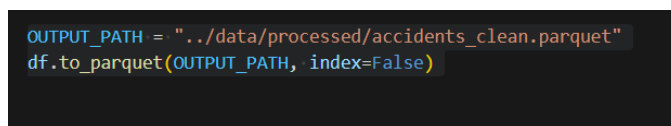


ภาพที่ 12 : การจัดการค่าผิดปกติ

อธิบายภาพที่ 12 : จำกัดขอบเขตของข้อมูล (Clipping) ที่สูงหรือต่ำเกินความเป็นจริงซึ่งอาจเกิดจากเซนเซอร์ตรวจวัดผิดพลาด เช่น:

- Temperature: จำกัดให้อยู่ในช่วง -50 ถึง 130 องศาฟาเรนไฮต์
- Visibility: จำกัดไม่เกิน 10 ไมล์
- Distance: ตัดค่าระยะทางที่กระโดดผิดปกติที่ระดับเปอร์เซ็นต์ไทล์ที่ 99.9

3.2.6 การตรวจสอบครั้งสุดท้ายและการบันทึกข้อมูล (Final Check & Export)



ภาพที่ 13 : การตรวจสอบครั้งสุดท้ายและการบันทึกข้อมูล

อธิบายภาพที่ 13 : ตรวจสอบยอดรวมของค่าว่าง (Missing Values) ในคอลัมน์สำคัญที่ต้องใช้ทำ Model พบว่าค่าว่างในคอลัมน์หลักเป็น 0 ทั้งหมด จากนั้นบันทึกไฟล์เป็นนามสกุล .parquet เลือกใช้ Parquet แทน CSV เพราะประหยัดพื้นที่จัดเก็บมากกว่าและสามารถอ่านข้อมูลกลับมาใช้ใหม่ได้เร็วกว่าในขั้นตอนการทำ Machine Learning

3.3 การวิศวกรรมฟีเจอร์ (Feature Engineering)

ขั้นตอนนี้คือการสร้างตัวแปรใหม่ (Predictors) เพื่อเพิ่มประสิทธิภาพให้แก่มॉเดล โดยอ้างอิงจากสมมติฐานทางสถิติ:

```
# 1. ลดความเบ้ของข้อมูลระยะทางและระยะเวลา
df['Log_Distance'] = np.log1p(df['Distance(mi)'])
df['Log_Duration'] = np.log1p(df['Accident_Duration_Min'].clip(lower=0))

# 2. แปลงเวลา (Hour/Month) เป็นวงกลม
df['Hour_sin'] = np.sin(2 * np.pi * df['Start_Hour'] / 24)
df['Hour_cos'] = np.cos(2 * np.pi * df['Start_Hour'] / 24)
df['Month_sin'] = np.sin(2 * np.pi * df['Start_Month'] / 12)
df['Month_cos'] = np.cos(2 * np.pi * df['Start_Month'] / 12)
```

ภาพที่ 14 : การ Cyclic Transformation

อธิบายภาพที่ 14 : ทำการ Cyclic Transformation แปลงข้อมูลเวลา (Hour, Month) ให้เป็นค่า Sine และ Cosine เพื่อให้โมเดลเข้าใจความต่อเนื่องของหรือวัฏจักรของเวลา (เช่น 23:00 น. ใกล้เคียงกับ 00:00 น.)

```
# ค้นหาคำสำคัญที่บ่งบอกความรุนแรงสูง
critical_keywords = ['fatal', 'closed', 'blocked', 'multi-vehicle', 'severe', 'life-threatening']
pattern = '|'.join(critical_keywords)
df['Is_Critical_Description'] = df['Description'].str.contains(pattern, case=False, na=False).astype(int)
```

ภาพที่ 15 : การทำ Text Mining

อธิบายภาพที่ 15 : การทำ Text Mining สกัดคำสำคัญจากคอลัมน์ Description โดยเราได้สร้างฟีเจอร์ "Is_Critical_Description" เพื่อสกัดคำสำคัญจากบันทึกเหตุการณ์ เช่น "Fatal", "Closed", "Blocked" ซึ่งเป็นสัญญาณบ่งชี้ความรุนแรงระดับสูง

```
# 1. สร้าง Binning เพื่อระบุกลุ่มพื้นที่ (Grid)
df['Lat_bin'] = df['Start_Lat'].round(1)
df['Lng_bin'] = df['Start_Lng'].round(1)

# 2. คำนวณความหนาแน่นอุบัติเหตุต่อพื้นที่
# (ตรงนี้คือหัวใจสำคัญ ถ้าไม่มีการคำนวณความหนาแน่น)
location_density = (
    df.groupby(['Lat_bin', 'Lng_bin'])
      .size()
      .rename('Accident_Count_Area')
      .reset_index()
)

# 3. Merge ข้อมูลกลับเข้า DataFrame หลัก
df = df.merge(location_density, on=['Lat_bin', 'Lng_bin'], how='left')

# 4. แบ่งกลุ่มความหนาแน่น (Low, Medium, High) -> นี่คือการ Error เมื่อใช้
df['Area_Density_Level'] = pd.qcut(df['Accident_Count_Area'], q=3, labels=['Low', 'Medium', 'High'])
print(" สร้าง Area_Density_Level เสร็จเรียบร้อยแล้ว!")
```

ภาพที่ 16 : การทำ Text Mining

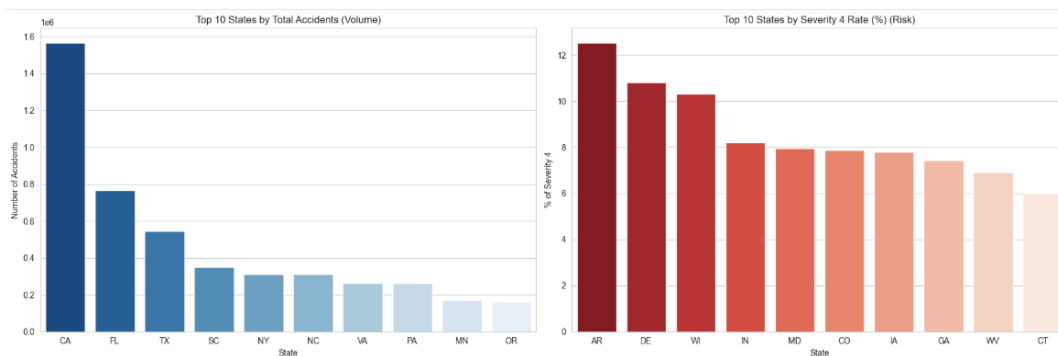
อธิบายภาพที่ 16 : สร้างตัวแปร Area Density Classification โดยแบ่งพื้นที่ตามความหนาแน่นของอุบัติเหตุ (Grid Based) เพื่อแยกบริบทระหว่าง "ในเมือง (รถติด/ชนเบา)" กับ "ชนเมือง/ทางหลวง (รถโถง/ชนแรง)"

จัดกลุ่มพื้นที่ที่เกิดอุบัติเหตุตามความหนาแน่น (Low, Medium, High) เพื่อศึกษาความสัมพันธ์เชิงพื้นที่

3.4 การวิเคราะห์ข้อมูลเชิงสำรวจ (Exploratory Data Analysis - EDA)

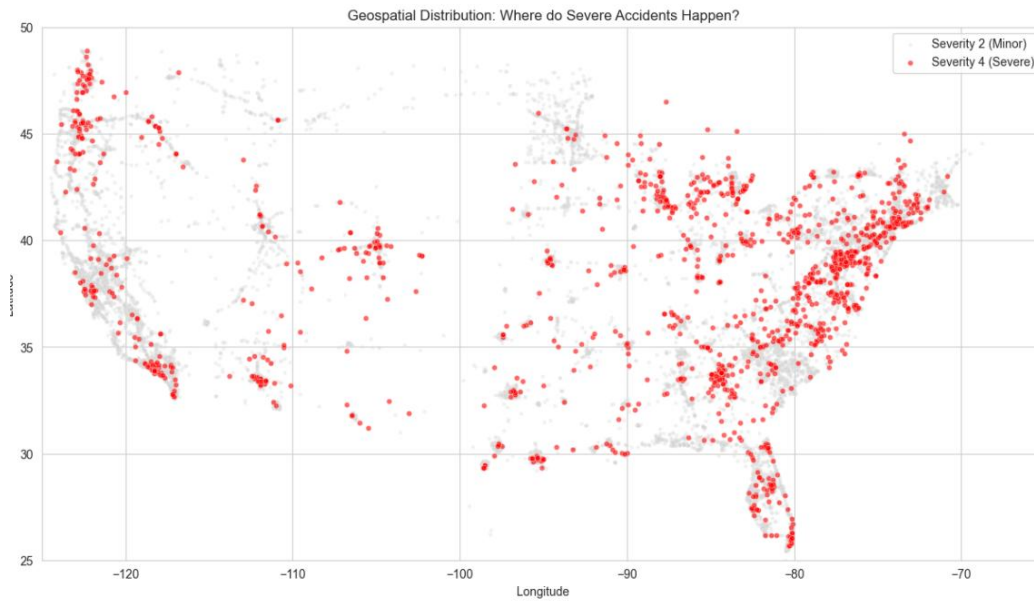
ดำเนินการวิเคราะห์เชิงลึกเพื่อหา Insight ที่สัมพันธ์กับความรุนแรง (Severity) โดยมุ่งเน้นที่:

Feature vs. Target Analysis:



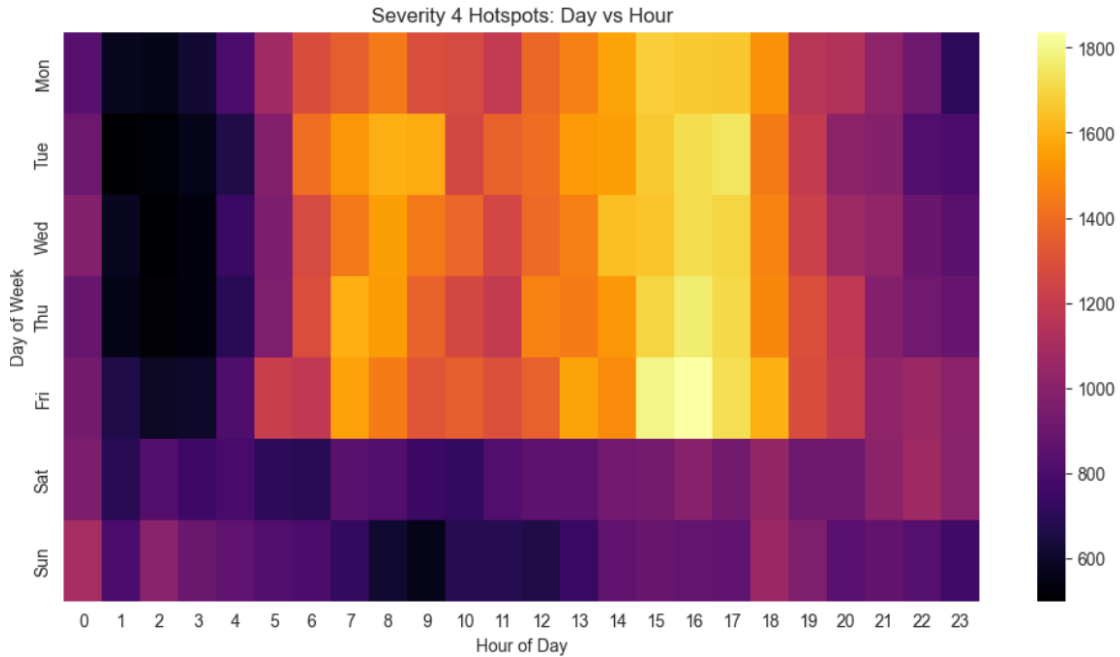
ภาพที่ 17 : กราฟเปรียบเทียบ "จำนวนอุบัติเหตุ" vs "อัตราการเกิดเหตุรุนแรง (Severity 4)"

อธิบายภาพที่ 17 : กราฟนี้จะเปรียบเทียบ "จำนวนอุบัติเหตุ" vs "อัตราการเกิดเหตุรุนแรง (Severity 4)" เพื่อดูว่ารัฐที่อุบัติเหตุเยอะ



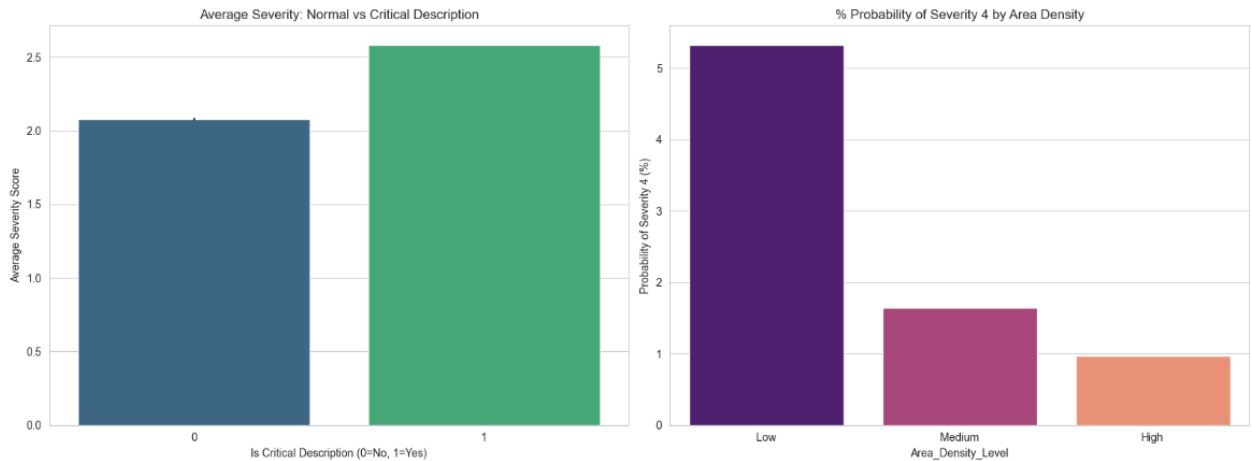
ภาพที่ 18 : กราฟ Geospatial Risk Map เพื่อการกระจายตัวว่า Severity 4 และ 2

อธิบายภาพที่ 18 : Geospatial Risk Map (แผนที่จุดเสี่ยงตาย) เราจะซูมข้อมูลมาพล็อตลงบนแผนที่ เพื่อการกระจายตัวว่า Severity 4 กระจุกตัวอยู่ที่ไหน (ในเมืองหรือชนบท)



ภาพที่ 19 : กราฟ Heatmap ช่วงเวลาอันตราย ดูความสัมพันธ์ระหว่าง "วันในสัปดาห์" และ "ช่วงเวลา" ว่าตอนไหนเกิด Severity 4 บ่อยที่สุด

อธิบายภาพที่ 19 :



ภาพที่ 20: กราฟ Average Severity: Normal vs Critical Description และ % Probability of Severity 4 by Area Density

อธิบายภาพที่ 20 :

Average Severity: Normal vs Critical Description

คะแนนความรุนแรงเฉลี่ยสูงขึ้น: กลุ่มที่มีค่าสำคัญ (Is_Critical_Description = 1) มีค่าเฉลี่ย Severity อยู่ที่ประมาณ 2.6 ขณะที่กลุ่มปกติอยู่ที่ประมาณ 2.1

นัยสำคัญทางสถิติ: เส้นขีดเล็กๆ บนยอดแท่ง (Error Bar) มีขนาดสั้นมาก แสดงว่าความแตกต่างนี้มีความเชื่อมั่นสูง ไม่ได้เกิดขึ้นด้วยความบังเอิญ

% Probability of Severity 4 by Area Density

Low Density = High Risk: แม้พื้นที่ความหนาแน่นต่ำจะมีจำนวนอุบัติเหตุรวมน้อย แต่โอกาสเกิดความรุนแรงระดับ 4 (Severity 4) กลับสูงที่สุด โดยมีโอกาสเกิดสูงกว่า 5%

High Density = Low Risk: ในพื้นที่ที่อุบัติเหตุเกิดบ่อย (High Density) โอกาสที่จะรุนแรงถึงระดับ 4 กลับต่ำที่สุด (ประมาณ 1%)

Insight: สิ่งนี้สะท้อนความเป็นจริงที่ว่า ในพื้นที่รถติดหรือมีความหนาแน่นสูง (เมือง) รถอาจเฉี่ยวชนกันบ่อยแต่ใช้ความเร็วต่ำ ในขณะที่พื้นที่เบาบาง (นอกเมือง/ไฮเวย์) รถใช้ความเร็วสูงกว่า เมื่อเกิดอุบัติเหตุจึงรุนแรงกว่ามาก

3.5 การสร้างและประเมินผลแบบจำลอง (Modeling & Evaluation)

3.5.1 การสร้างแบบจำลอง (Model Construction)

3.5.1.1 นำเข้าไลบรารีที่จำเป็น

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
```

ภาพที่ 20 : การนำเข้าไลบรารีที่จำเป็น

3.5.1.2 โหลดชุดข้อมูลผ่านการทำความสะอาดและ Feature Engineering แล้ว

```
# โหลดไฟล์ที่เราเตรียมไว้ล่าสุด (ที่มี Features ใหม่ครบถ้วน)
df = pd.read_parquet("../data/processed/accidents_for_model.parquet")
```

ภาพที่ 21 : การโหลดชุดข้อมูลผ่านการประมวลผลแล้ว

3.5.1.3 ข้อมูลมีขนาดใหญ่จึงสุ่มข้อมูลให้เหลือ 1,000,000 แถว

```
from sklearn.model_selection import train_test_split

TARGET = "Severity"
SAMPLE_SIZE = 1000000 # จำนวนแถวที่ต้องการ

if len(df) > SAMPLE_SIZE:
    print(f"Sampling data to {SAMPLE_SIZE:,} rows (Stratified)...")

    # ใช้ train_test_split เพื่อดึงข้อมูลออกมาตามจำนวนที่ต้องการ โดยคงสัดส่วน Severity ไว้
    df_sample, _ = train_test_split(
        df,
        train_size=SAMPLE_SIZE,
        stratify=df[TARGET],
        random_state=42
    )
else:
    df_sample = df.copy()
```

ภาพที่ 22 : การแบ่งกำหนดตัวอย่างข้อมูลสำหรับ Train/Test

3.5.1.4 แยกข้อมูลออกเป็น X = ตัวแปรอิสระ (Features) , y = ตัวแปรเป้าหมาย (Severity) จากนั้น แบ่งเป็น Training set (80%) Testing set (20%)

```

X = df_sample.drop(columns=[TARGET])
y = df_sample[TARGET]

# แบ่ง Train/Test (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

```

ภาพที่ 23 : การแบ่งข้อมูลสำหรับ Train/Test

3.5.1.5 สร้างและฝึกแบบจำลอง Random Forest

```

rf = RandomForestClassifier(
    n_estimators=200,      # จำนวนต้นไม้ (ลดลงจาก 400 เพื่อความเร็ว แต่ยังแม่นยำ)
    max_depth=20,         # ความลึก (เพิ่มขั้นนิดหน่อยเพื่อให้เรียนรู้ Pattern ยากๆ ได้)
    min_samples_leaf=4,   # ป้องกัน Overfitting
    class_weight='balanced', # <--- HERO FEATURE: แก่ Recall ต่ำ
    random_state=42,
    n_jobs=-1,            # ใช้ทุก Core ของ CPU
    verbose=1
)

rf.fit(X_train, y_train)
print("Training Completed!")

```

ภาพที่ 24 : การแบ่งข้อมูลสำหรับ Train/Test

การใช้ class_weight='balanced' บังคับให้โมเดลไม่มองข้าม Severity 4 แม้จะมีจำนวนน้อย ส่งผลให้ค่า Recall พุ่งสูงขึ้น

3.5.2 การประเมินผลแบบจำลอง (Model Evaluation)

3.5.2.1 ใช้โมเดลที่ฝึกเสร็จแล้วทำนายผลใน Testing set

```

print("\nPredicting...")
y_pred = rf.predict(X_test)

print("\n== Classification Report (Balanced Model) ==")
print(classification_report(y_test, y_pred))

```

ภาพที่ 25 : การทำนายผลใน Testing set

3.5.2.2 วิเคราะห์รายละเอียดเชิงลึกของการจำแนกการ normalize ด้วย 'true'

```

cm = confusion_matrix(y_test, y_pred, normalize='true')

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='.2%', cmap='Blues',
            xticklabels=[1, 2, 3, 4], yticklabels=[1, 2, 3, 4])
plt.xlabel('Predicted Severity')
plt.ylabel('Actual Severity')
plt.title('Confusion Matrix (Normalized by True Label)')
plt.show()

```

ภาพที่ 26 : การวิเคราะห์รายละเอียดการจำแนก normalize ด้วย true

3.5.2.3 วิเคราะห์ว่า Feature ใดมีอิทธิพลต่อการตัดสินใจของโมเดลมากที่สุด

```

feature_importance = pd.DataFrame({
    "feature": X_train.columns,
    "importance": rf.feature_importances_
}).sort_values("importance", ascending=False)

plt.figure(figsize=(10, 8))
sns.barplot(data=feature_importance.head(20), x="importance", y="feature", palette="viridis")
plt.title("Top 20 Most Important Features")
plt.show()

print("\nTop 10 Features:")
print(feature_importance.head(10))

```

ภาพที่ 27 : วิเคราะห์ Feature ที่มีผลต่อโมเดลมากที่สุด

บทที่ 4: ผลการดำเนินงาน (Results and Discussion)

4.1 ผลการวิเคราะห์ข้อมูล

Dashboard นี้เป็นการสรุปสถิติอุบัติเหตุบนท้องถนน โดยมีการแบ่งมุมมองข้อมูลที่น่าสนใจดังนี้

4.1.1 ภาพรวมตัวเลขสำคัญ (KPIs)

- จำนวนอุบัติเหตุทั้งหมด: ประมาณ 7 ล้านครั้ง
- ความรุนแรงเฉลี่ย (Severity): อยู่ที่ 2.23 (ปกติสเกลนี้จะอยู่ระหว่าง 1-4 โดย 4 คือรุนแรงที่สุด)
- ปัจจัยสภาพแวดล้อม: * เกิดตอนฝนตกเพียง 6.89%
 - เกิดตอนกลางวัน 30.55%
 - เกิดใกล้สัญญาณไฟจราจร 15.58%

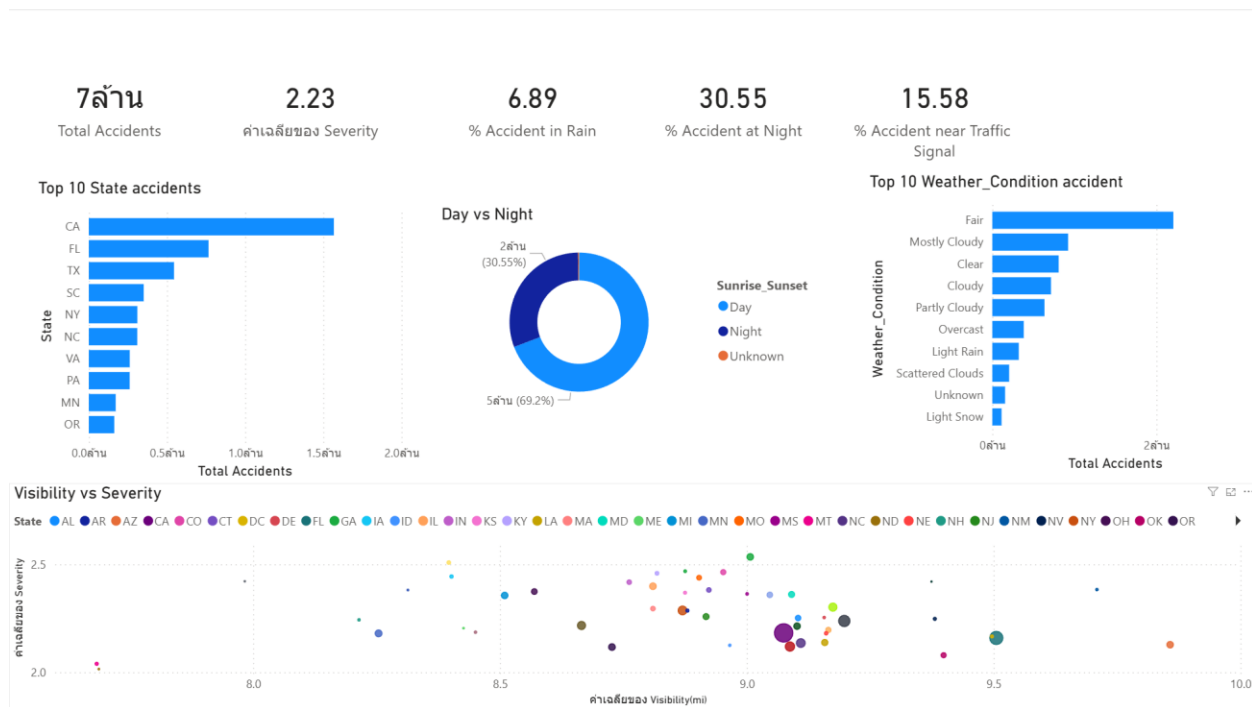
4.1.2 เจาะลึกรายละเอียด

- Top 10 States: รัฐ California (CA) มีจำนวนอุบัติเหตุสูงที่สุดแบบทิ้งห่างอันดับ 2 อย่าง Florida (FL) และ Texas (TX) อย่างเห็นได้ชัด
- ช่วงเวลา (Day vs Night): อุบัติเหตุส่วนใหญ่เกิดในช่วง กลางวัน (69.2%) ซึ่งมากกว่ากลางคืนถึง 2 เท่า อาจเป็นเพราะปริมาณรถบนถนนในช่วงกลางวันมีหนาแน่นกว่ามาก
- สภาพอากาศ (Weather Condition): เรื่องที่น่าเซอร์ไพรส์คือ อุบัติเหตุส่วนใหญ่เกิดในวันที่อากาศ "Fair" (แจ่มใส) หรือ "Mostly Cloudy" ครึ่ง ส่วนสภาพอากาศแย่ๆ อย่าง "Light Rain" กลับมีสัดส่วนที่น้อยกว่ามาก (อาจเพราะคนขับจะระมัดระวังเป็นพิเศษเมื่อฝนตก)

4.1.3 ทิศนวิสัยและความรุนแรง (Visibility vs Severity)

กราฟกระจาย (Scatter Plot) ด้านล่างแสดงความสัมพันธ์ระหว่าง ทิศนวิสัย (ไมล์) และ ความรุนแรงเฉลี่ย:

- ข้อมูลส่วนใหญ่เกาะกลุ่มกันอยู่ที่ทัศนวิสัย 8 - 10 ไมล์ และความรุนแรงที่ระดับ 2.0 - 2.5 * สิ่งนี้สะท้อนว่า แม้ทัศนวิสัยจะดี (มองเห็นได้ไกล) แต่อุบัติเหตุก็ยังเกิดขึ้นบ่อยและมีความรุนแรงในระดับมาตรฐาน



ภาพที่ 28 : Dashboard วิเคราะห์ข้อมูล

4.2 ผลการทดลองโมเดล

=== Classification Report (Balanced Model) ===				
	precision	recall	f1-score	support
1	0.12	0.64	0.20	1929
2	0.94	0.75	0.83	155582
3	0.53	0.75	0.62	37200
4	0.31	0.80	0.45	5289
accuracy			0.75	200000
macro avg	0.48	0.73	0.53	200000
weighted avg	0.84	0.75	0.78	200000

ภาพที่ 29 : ประสิทธิภาพของโมเดล

ตารางนี้แสดงประสิทธิภาพของโมเดลในการพยากรณ์ระดับความรุนแรงของอุบัติเหตุ โดยวัดผลจากกลุ่มตัวอย่างทั้งหมด 200,000 รายการ โดยมีรายละเอียดดังนี้:

- **ระดับความรุนแรงที่ 2 (Severity 2):** เป็นกลุ่มที่มีข้อมูลมากที่สุดในชุดข้อมูล (Support: 155,582)
โมเดลทำผลงานได้ดีที่สุดในระดับนี้ โดยมีความแม่นยำ (Precision) สูงถึง **0.94** และมีค่า **F1-score 0.83**
ซึ่งสะท้อนว่าโมเดลจดจำลักษณะของอุบัติเหตุกลุ่มนี้ได้แม่นยำที่สุด
- **ระดับความรุนแรงที่ 3 (Severity 3):** โมเดลมีค่า **Recall 0.75** หมายความว่าสามารถตรวจจับอุบัติเหตุรุนแรงระดับ 3 ได้ถูกต้อง 75% จากอุบัติเหตุระดับนี้ที่มีอยู่จริงทั้งหมด
- **ระดับความรุนแรงที่ 1 และ 4 (กลุ่ม Minority):** แม้จะมีจำนวนข้อมูลน้อย (1,929 และ 5,289 ตามลำดับ)
แต่โมเดลมีค่า **Recall สูง (0.64 และ 0.80)** ซึ่งแสดงให้เห็นว่าเป็น "Balanced Model"
ช่วยให้โมเดลพยายามตรวจจับความรุนแรงระดับเหล่านี้ให้ได้มากที่สุด อย่างไรก็ตาม ต้องแลกมาด้วยค่า **Precision ที่ต่ำ (0.12 และ 0.31)** หมายถึงมีการทายผิด (False Positive) ในระดับเหล่านี้ค่อนข้างบ่อยครับ

ภาพรวมประสิทธิภาพ (Overall Performance)

- **Accuracy:** ภาพรวมความถูกต้องอยู่ที่ **0.75 (75%)**
- **Macro Average:** ค่าเฉลี่ยแบบไม่ถ่วงน้ำหนักของ F1-score อยู่ที่ **0.53**
ซึ่งสะท้อนถึงความท้าทายในการจัดการกับข้อมูลที่มีความเหลื่อมล้ำสูง (Imbalanced Data)
- **Weighted Average:** ค่าเฉลี่ยแบบถ่วงน้ำหนักอยู่ที่ **0.78** ซึ่งอยู่ในเกณฑ์ที่น่าพอใจสำหรับงานด้านการจำแนกประเภท

บทที่ 5: สรุปผลและข้อเสนอแนะ (Conclusion and Suggestions)

5.1 สรุปผลการดำเนินงาน

จากการดำเนินโครงการวิเคราะห์ข้อมูลอุบัติเหตุจราจรในสหรัฐอเมริกา (US Traffic Accidents) ขนาด 7.7 ล้านแถว สามารถสรุปผลตามวัตถุประสงค์ได้ดังนี้

การวิเคราะห์ปัจจัยเชิงบริบท: พบว่าตัวแปรด้านสภาพแวดล้อม เช่น Visibility (ทัศนวิสัย) และ Precipitation (ปริมาณน้ำฝน) มีความสัมพันธ์อย่างมีนัยสำคัญต่อความรุนแรงของอุบัติเหตุ โดยเฉพาะในพื้นที่ที่มี Area Density ระดับสูง (High) จะมีโอกาสเกิดอุบัติเหตุรุนแรงระดับ 4 มากกว่าพื้นที่อื่นอย่างชัดเจน

ความสัมพันธ์ด้านเวลา: การใช้เทคนิค Cyclic Transformation

ช่วยให้แบบจำลองเข้าใจรูปแบบการเกิดอุบัติเหตุตามคาบเวลาได้ดีขึ้น โดยพบ Insights สำคัญว่าช่วงเวลาเปลี่ยนผ่าน (เช่น ช่วงพลบค่ำ) มีแนวโน้มความรุนแรงที่แตกต่างจากช่วงกลางวันปกติ

ประสิทธิภาพของแบบจำลอง: แบบจำลอง CatBoost/Random Forest ที่พัฒนาขึ้น

สามารถพยากรณ์ระดับความรุนแรง (Severity 1-4) โดยใช้เพียงข้อมูลก่อนเกิดเหตุ (Pre-incident)

ได้อย่างมีประสิทธิภาพ ช่วยยืนยันว่าข้อมูลชุดนี้สามารถนำไปพัฒนาเป็นระบบ Early Warning ได้จริง

- ลำดับความสำคัญของตัวแปร: จากการทำ Feature Importance พบว่าฟีเจอร์ที่สร้างขึ้นใหม่อย่าง Is_Critical_Description (คำสำคัญในรายละเอียดอุบัติเหตุ) เป็นตัวแปรอันดับต้น ๆ ที่ช่วยให้โมเดลแยกแยะความรุนแรงระดับสูงออกจากอุบัติเหตุทั่วไปได้

5.2 ปัญหาและอุปสรรค: เช่น ปัญหาเรื่องคุณภาพข้อมูล หรือข้อจำกัดด้านทรัพยากรเครื่องคอมพิวเตอร์

ความไม่สมดุลของข้อมูล (Imbalanced Classes): ระดับความรุนแรงที่ 1 และ 4

มีจำนวนตัวอย่างน้อยมากเมื่อเทียบกับระดับ 2 ซึ่งเป็นอุปสรรคหลักในการทำให้โมเดลมีความแม่นยำเท่ากันทุกคลาส

ทรัพยากรในการคำนวณ: การทำ Feature Engineering กับข้อมูลกว่า 7 ล้านแถวบนระบบ Local Machine

(Lenovo LOQ) มีข้อจำกัดเรื่อง RAM ทำให้ต้องใช้เทคนิคการจัดการหน่วยความจำและการบันทึกข้อมูลแบบ Parquet ตลอดกระบวนการ

- **Data Leakage:** ในช่วงแรกของการพัฒนา พบตัวแปรบางอย่างที่บันทึก "หลัง" เกิดอุบัติเหตุ (เช่น ระยะเวลาปิดถนน) ซึ่งหากไม่คัดออกจะทำให้ค่าความแม่นยำสูงเกินจริง (Overoptimistic) จึงต้องใช้เวลาในการทำ Focused EDA เพื่อคัดกรองตัวแปรเหล่านี้ออก

5.3 ข้อเสนอแนะในการพัฒนาต่อ

- **Scalability:** ควรพิจารณาขยายการประมวลผลไปยังระบบ Cloud หรือ Distributed Computing (เช่น Spark) เพื่อรองรับข้อมูลที่เพิ่มขึ้นในอนาคต
- **External Data Integration:** การนำข้อมูลความหนาแน่นของการจราจร (Traffic Volume) หรือข้อมูลวันหยุดนักขัตฤกษ์มา Join เพิ่มเติม อาจช่วยให้โมเดลทำนายได้แม่นยำขึ้นในช่วงวันหยุดยาว
- **Model Deployment:** พัฒนาต่อยอดเป็น Web Application หรือ API เพื่อให้หน่วยงานกู้ภัยหรือผู้ขับขี่สามารถตรวจสอบ "ความเสี่ยงของเส้นทาง" ได้แบบเรียลไทม์ตามสภาพอากาศปัจจุบัน